作业 HW2 实验报告

姓名：于沛可 学号：2451951 日期：2025 年 10 月 19 日

1. 涉及数据结构和相关背景

栈（Stack）是一种遵循“后进先出”原则的线性结构，允许在同一端进行插入和删除操作。其基本操作包括：初始化、判空、压栈、弹栈、获取栈顶元素、清空栈以及返回栈内元素的数量。

根据实现方式不同，栈可以分为顺序栈和链栈两种：

1. **顺序栈** 使用固定大小的数组实现，需要预先定义栈容量，容易出现上溢（栈满时尝试压栈）和下溢（栈空时尝试弹栈）的问题。

2. **链栈** 采用链表结构实现，支持动态扩展，无需预设容量，栈顶指针始终指向链表头部，操作更为灵活。

队列（Queue）是一种遵循“先进先出”原则的线性结构，允许在一端插入元素、在另一端删除元素。其基本操作包括：初始化、销毁队列、入队、出队、判空以及获取队头元素。

队列根据实现方式不同，可以分为顺序队列、循环队列和链队列三类：

1. **顺序队列** 使用数组实现，队头和队尾指针分别管理队列元素；当两者相等时表示队列为空，当队尾指针到达最大长度时表示队列已满。

2. **循环队列** 将数组首尾相连，通过取模运算管理队头队尾指针，减少空间浪费，但需保留一个空位区分“队满”与“队空”状态。

3. **链队列** 基于链表结构实现，动态扩展，队头和队尾各有指针，方便高效管理。

栈与队列广泛应用于计算机科学与编程中。栈常用于函数调用、表达式求值、括号匹配等需要“回溯”或“临时存储”的场景；队列则常用于任务调度、打印任务管理、图的广度优先搜索（BFS）等需要保持处理顺序的场景。根据问题性质合理选择栈或队列，可以显著提升算法实现的效率与可读性。

2. 实验内容

2.1 列车入栈

2.1.1 题目描述

给定一个固定的列车入站序列，判断若干个目标出站序列是否可以通过以下三种操作组合实现：

1.列车从入口直接进入出口（a）

2.列车从入口进入车站（b）

3.列车从车站进入出口（c）

2.1.2 基本要求

根据给定的入站序列和多组出站序列，判断每组出站序列是否可以由栈的 LIFO 特性模拟得出。如果可行，输出 "yes"，否则输出 "no"。输入结束标志为 EOF。

2.1.3 数据结构设计

```
#define STACK_INIT_SIZE 100 // 栈初始分配空间大小
#define STACKINCREMENT 10  // 栈满时增加的存储空间
```

```
struct Stack
{
    char *base; //栈尾指针
    char *top;  //栈顶指针
    int stacksize;  //当前已分配的存储空间，单位是sizeof(元素)
};
```

2.1.4 功能说明

2.1.4.1 栈初始化函数

```
/**
 * @brief 初始化顺序栈S
 * @param S 待初始化的栈引用
 * @return int 1表示成功，exit(1)表示失败
 */
int InitStack(Stack &S)  //栈初始化
{
    S.base=(char*)malloc(STACK_INIT_SIZE*sizeof(char));
    if(S.base==NULL) exit(1);
    S.top=S.base;
    S.stacksize=STACK_INIT_SIZE;
    return 1;
}
```

2.1.4.2 插入元素为新的栈顶元素

```
/**
 * @brief 元素入栈
 * @param S 栈引用
 * @param e 待入栈元素
 * @return 1表示成功
 */
int Push(Stack &S, char e) {
    若 栈已满
        重新分配更大空间（realloc）
        更新 top 指针
    *S.top = e
    S.top++
    return 1
}
/**
```

2.1.4.3 删除栈顶元素并返回其值

```
 * @brief 删除栈顶元素并返回其值
 * @param S 栈引用
 * @param e 接收出栈元素
 * @return 1成功，0失败
 */
int Pop(Stack &S, char &e) {
    若 S 为空
        return 0
    S.top--
    e = *(S.top)
    return 1
}
```

2.1.4.4 模拟出栈函数（核心）

```
/**
 * @brief 判断出站序列是否可由入站序列得到
 * @param str_in 入站序列
 * @param str_out 出站序列
 * @param S 栈（模拟车站）
 * @return true若可行，否则false
 */
```

```
bool check_sequence(string str_in, string str_out, Stack &S) {
    清空并初始化栈 S
    若 两序列长度不同
        return false
    i = 0   // 入站指针
    j = 0   // 出站指针
    char e
    while (j < str_out.length()) {
        // 阶段1：若栈顶不满足期望车，则继续入站
        while (i < str_in.length() 且 (栈为空 或 栈顶 != str_out[j])) {
          s     若 str_in[i] == str_out[j]:
                // 操作A：入口直接出站
                i++, j++
                跳到下一轮外层循环
              否则:
                // 操作B：入口进栈
                Push(S, str_in[i])
                i++
        }
        // 阶段2：若栈顶满足出站车，执行出栈（操作C）
        若 栈非空 且 栈顶 == str_out[j]:
            Pop(S, e)
            j++
            continue
        // 阶段3：若入口耗尽且栈顶不匹配
        return false
    下一轮外层循环
    }
    return true
}
```

2.1.4.5 取栈顶元素

```
/**
 * @brief 获取栈顶元素
 * @param S 栈
 * @param e 引用接收栈顶元素
 * @return 1成功，0失败
 */
int GetTop(Stack S, char &e) {
    若 S 为空栈或不存在
        return 0
    e = *(S.top - 1)
    return 1
}
```

2.1.4.6 清空栈

```
/**
 * @brief 清空栈并释放内存
 * @param S 栈引用
 */
void ClearStack(Stack &S) {
    若 S.base 非空
        释放 S.base              // free 动态分配的存储空间
    S.base = NULL              // 重置指针与容量信息
    S.top = NULL
    S.stacksize = 0
}
```

2.1.4.7 判断栈是否为空

```
/**
 * @brief 判断栈是否为空
 * @param S 栈常量引用
 * @return bool true如果栈为空，false否则
 */
bool StackEmpty(const Stack &S) //判断栈是否为空
{
    return S.base==S.top;
}
```

2.1.4.8 判断某元素是否在栈里

```
/**
 * @brief 判断某字符是否在栈中
 * @param S 栈结构体
 * @param e 待查找的字符
 * @return 1表示在栈里，0表示不在
 */
int In(Stack S, char e) {
    p = S.base                    // 从栈底开始扫描
    while (*p != e 且 p < S.top)
        p++                       // 逐个比较栈中元素
    若 p != S.top
        return 1                  // 找到目标字符
    否则
        return 0                  // 未找到
}
```

2.1.5 调试分析

1. 栈的重复初始化问题：入栈序列 str_in 是固定的，但是 str_out 是多组，因此在每次 str_out 合理性判断时，必须在 check_sequence 函数头部加入栈的清空与初始化操作。

2. Check_sequence 核心函数的设计（难点）

起初我设计此函数时，条件语句与循环语句混杂极为混乱，后来引入贪心策略，并设计了三阶段流程：

阶段 1（引入新车/操作 A，B）

阶段 2（车站出站/操作 C）

阶段 3（失败判定），采用双层 while 循环判断，尤其是内层循环的条件设计要谨慎考虑。

2.1.6 总结和体会

1. 通过本题强化了对顺序栈的设计与应用到实际问题的认识。

2. 通过本题，进一步强化双指针方法的应用（入栈序列和出栈序列的双指针），并且学习了贪心算法的思想：通过局部最优解得到全局最优解的思想。在此题中表现为：只要能让当前期望的列车出站（无论是从栈顶还是从入口），就立刻执行；否则，就把当前入口列车推进栈，为后面的正确列车清出通道。

2.2 布尔表达式

2.2.1 题目描述

计算如下布尔表达式（V | V）& F &（F | V） 其中 V 表示 True，F 表示 False，|表示 or，&表示 and，！表示 not（运算符优先级 not> and > or）

2.2.2 基本要求

文件输入，有若干（A<=20）个表达式，其中每一行为一个表达式。 表达式有（N<=100）个符号，符号间可以用任意空格分开，或者没有空格，所以表达式的总长度，即字符的个数，是未知的。

对于 20%的数据，有 A<=5，N<=20，且表达式中包含 V、F、&、|

对于 40%的数据，有 A<=10，N<=50，且表达式中包含 V、F、&、|、！

对于 100%的数据，有 A<=20，N<=100，且表达式中包含 V、F、&、|、!、(、)

2.2.3 数据结构设计

```
#define MAXSIZE 200 // 根据题意，支持 100 长度表达式足够
struct Stack //静态栈
{
    char data[MAXSIZE];
    int top; // 指向当前栈顶的下一个位置
};
```

2.2.4 功能说明

2.2.4.1 初始化函数

```
 * @brief 初始化栈
 * @param S  栈引用
 */
void InitStack(Stack &S) {
    S.top = 0;
}
```

2.2.4.2 比较运算符优先级函数

```
/**
 * @brief 比较两个运算符优先级
 * @param theta1 栈顶运算符
 * @param theta2 当前读入运算符
 * @return '<' 表示当前运算符入栈
 *         '>' 表示栈顶运算符出栈计算
 *         '=' 表示括号匹配或结束符
 */
char Precede(char theta1, char theta2) {
    if ((theta1=='('&&theta2==')')||(theta1=='#'&&theta2=='#')) return '=';
    else if (theta1=='(' || theta1=='#' || theta2=='(') return '<';
    else if (theta2==')') return '>';
    else if (theta1=='!' && theta2=='!') return '<'; // !右结合
    // 其余比较按优先级数值
    ...
}
```

2.2.4.3 对运算数进行运算的函数

```
/**
 * @brief 执行一次逻辑运算
 * @param first  左操作数
 * @param theta  运算符
 * @param second 右操作数
 * @return 运算结果 ('V' 或 'F')
 */
char Operate(char first, char theta, char second) {
    switch (theta) {
        case '|': return (first=='F'&&second=='F')?'F':'V';
        case '&': return (first=='V'&&second=='V')?'V':'F';
        case '!': return (second=='V')?'F':'V';
    }
}
```

2.2.4.4 运算表达式值的函数（核心）

```
 * @brief 评估一个布尔表达式
 * @param expr 表达式字符串
 * @return 'V' 或 'F'
 */
char EvaluateExpression(string expr) {
    运算符栈 S1, 操作数栈 S2
    Push(S1, '#')
    去除空格并在末尾添加 '#'
    i = 0, ch = expr[i++]
    while (ch != '#' || 栈顶(S1) != '#') {
        if (ch 是 'V' 或 'F')
            Push(S2, ch)
        else if (ch 是运算符) {
            比较 Precede(栈顶(S1), ch)
            若结果为 '<': Push(S1, ch)
            若结果为 '=': Pop(S1)
            若结果为 '>': Pop(S1, e), 从 S2 弹出一个或两个操作数计算，再压回结果
        }
        取下一个字符 ch
    }
    返回 栈顶(S2)
}
```

2.2.4.4 元素入栈函数

```
/**
 * @brief 元素入栈
 * @param S 栈引用
 * @param ch 压入元素
 */
bool Push(Stack &S, char ch) {
    if (S.top >= MAXSIZE) return false;
    S.data[S.top++] = ch;
    return true;
}
```

2.2.4.5 元素出栈函数

```
/**
 * @brief 弹出栈顶元素
 * @param S 栈引用
 * @param e 接收弹出元素
 */
bool Pop(Stack &S, char &e) {
    if (S.top == 0) return false;
    e = S.data[--S.top];
    return true;
}
```

2.2.4.6 获取栈顶元素函数（不弹出）

```
/**
 * @brief 获取栈顶元素但不弹出
 */
bool GetTop(Stack &S, char &e) {
    if (S.top == 0) return false;
    e = S.data[S.top - 1];
    return true;
}
```

2.2.4.7 判断字符是否应该进入运算符栈

```
/**
 * @brief 判断字符是否属于运算符集合
 */
bool In(char ch) {
    return (ch == '!' || ch == '&' || ch == '|' || ch == '(' || ch == ')' || ch
    == '#');
}
```

2.2.5 调试分析

1. 数据结构设计选择

在最初栈的实现中，我使用了传统的动态分配的顺序栈结构（通过 malloc 和 realloc 管理内存），在 TLE 运行时频繁分配和回收内存，导致测试点 TLE。

再次分析分析，发现对于固定长度（N ≤ 100）的表达式，动态扩容是完全没有必要的，因此改用静态数组栈结构。

2. ！作为单目运算符的处理

起初该题有一个测试点始终无法通过，后来分析是连续取反（!!V）计算错误：

原因是算符优先表未考虑 ！ 的右结合性，导致在处理连续 ！ 时提前弹栈。

通过在 Precede() 中增加一条判断 if (theta1==’!’ && theta2==’!’) return ’<’; 得以解决

3. 空栈弹出引发运行错误

起初该题有两个测试点始终 runtime error，猜测可能有操作数组越界等问题。

如：在某些情况下（如表达式开头是 ！），操作数栈可能为空。解决方法：在执行弹出前增加判空逻辑，若空则默认使用 'F'，既保证程序稳定又避免错误。

2.2.6 总结与体会

本实验让我更加理解数据结构与算法效率之间的关系。相同的逻辑结构（算符优先法），不同的物理存储实现方式，会导致性能和稳定性完全不同。

2.3 最长子串

2.3.1 题目描述

给定一个长度为 n，仅含有字符 '(' 和 ')' 的字符串，计算出最长的正确的括号子串的长度及起始位置。若存在多个最长子串，取第一个的起始位置。

2.3.2 基本要求

字符串长度：$0 \leqslant n \leqslant 1*10^5$

对于 20%的数据：0<=n<=20

对于 40%的数据：0<=n<=100

对于 60%的数据：0<=n<=10000

对于 100%的数据：0<=n<=100000

2.3.3 数据结构设计

```
#define STACK_INIT_SIZE 100 // 栈初始分配空间大小
#define STACKINCREMENT 10  // 栈满时增加的存储空间
struct Stack
{
    int *base; //栈尾指针
    int *top;  //栈顶指针
    int stacksize;  //当前已分配的存储空间，单位是sizeof(元素)
};
```

2.3.4 功能说明

此题中栈的初始化、将元素插入栈顶、删除栈顶元素并返回其值、返回栈顶元素值（不删除）、判断元素栈是否为空等基本函数与第一题中几乎一样，只是栈的元素又 char 改为 int 型，不再赘述。

2.3.4.6 求字符串中最长的合法括号子串（核心函数）

```
* @brief  求字符串中最长的合法括号子串
* @param  s        输入字符串（只含 '(' 和 ')'）
* @param  max_len  引用，返回的最大长度（调用前建议初始化为 0）
* @param  pos      引用，返回最长子串的起始下标（若无合法子串可设为 0 或 -1）
* 算法思想（栈存放下标）：
*  1) 在栈中先压入基准索引 -1（便于长度计算）
*  2) 从 i = 0 扫描到 n-1：
*       - 遇到 '(' ：将索引 i 入栈
*       - 遇到 ')' ：先尝试出栈（匹配一个 '('）
*           * 出栈后若栈为空：说明当前 ')' 没有匹配的 '(' → 将当前索引 i 压入栈，作为新的基准
*           * 否则：栈顶 top_val 为当前未匹配索引的前一个位置，合法长度 = i - top_val
*               如果合法长度 > max_len，则更新 max_len 和 pos = top_val + 1
```

```
Pair(s, max_len, pos):
    // 1. 空串处理
    if s.length == 0:
        max_len ← 0
        pos ← -1
        return
    // 2. 初始化栈并压入基准 -1
    创建空栈 S
    Push(S, -1)        // 基准索引，便于计算长度
    // 3. 扫描字符串
    for i from 0 to s.length - 1:
        if s[i] == '(':
            Push(S, i)
        else: // s[i] == ')'
            // 3.1 先尝试出栈（匹配前一个 '(' 或基准 -1）
            Pop(S, tmp)     // tmp 接收被弹出的值
            // Pop 可能返回 0（表示栈不存在或为空），需处理返回值以避免未定义行为
            // 3.2 如果出栈后栈为空，说明当前 ')' 没有匹配
            if StackEmpty(S):
                Push(S, i)            // 把当前位置作为新的基准
            else:
                // 3.3 否则读取栈顶索引 top_val（栈顶是未匹配位置的前一个索引）
                GetTop(S, top_val)
                curr_len ← i - top_val
                if curr_len > max_len:
                    max_len ← curr_len
                    pos ← top_val + 1
    return;
```

2.3.5 调试分析

1. 栈的设计

刚开始我将栈设为存放字符‘（’‘）’的栈，后来发现此种方式难以记录正确的最长子串长度以及起始位置（要用到更复杂的动态规划思想），因此我将栈设置为存放字符下标索引的栈，这样便于实现对最长子串长度以及起始位置的随时记录。

2. 难点：如何动态追踪最长子串

设计主要函数时必须同时处理两种边界：一是栈初始化时的虚拟边界，解决方法是把-1 压入栈；二是如何跟踪非法索引的动态边界，解决方法是当遇到多余的“）”导致栈为空时，重置边界，将当前非法的位置标记为下一段合法子串的起点，确保只计算连续的子串。

2.3.6 总结和体会

此题看似与上一道布尔运算很像，都是有关字符串，但栈中实际存放的元素类型非常不同，因此要根据实际情况来设计数据结构。

2.4 队列的应用

2.4.1 题目描述

输入一个 n*m 的 0 1 矩阵，1 表示该位置有东西，0 表示该位置没有东西。所有四邻域联通的 1 算作一个区域，仅在矩阵边缘联通的不算作区域。求区域数。此算法在细胞计数上会经常用到。对于所有数据，0<=n,m<=1000

2.4.2 基本要求

输入：

第 1 行 2 个正整数 n，m，表示要输入的矩阵行数和列数

第 2—n+1 行为 n*m 的矩阵，每个元素的值为 0 或 1。

输出：

1 行，代表区域数

2.4.3 数据结构设计

```
struct Node //链式队列的节点
{
    int x, y;       // 存储坐标
    Node* next;     // 指向下一个节点
};
struct Queue //链式队列
{
    Node* front;   // 队头指针（不存数据）
    Node* rear;    // 队尾指针
```

其余辅助数组说明：

```
const int MAXN = 1000; //题目保证n,m<=1000
bool Map[MAXN][MAXN];  // 存储输入的0/1地图
bool Vis[MAXN][MAXN];  // 访问标记，记录该坐标是否已被BFS访问过
int n, m;              // 行列数
//四个方向的偏移量，用于BFS搜索四领域
int dx[4] = {0, 0, -1, 1}; //行偏移：左右（0），上（-1），下（1）
int dy[4] = {-1, 1, 0, 0}; //列偏移：上下（0），左（-1），右（1）
```

2.4.4 功能实现

2.4.4.1Queue 相关基本函数简略说明

函数名	功能说明	设计逻辑
`init()`	初始化队列。`front = rear = new Node; front->next = NULL;`	创建一个**空的头结点**，让 `front` 和 `rear` 都指向它。此时 `front == rear` 表示队列为空。
`empty()`	判断队列是否为空。`return front == rear;`	队列为空的判断条件：**头指针和尾指针指向同一个（空头）节点。**
`push(x, y)`	入队：创建新节点 `p`，插入到 `rear` 后面，然后更新 `rear` 指向 `p`。	**队尾操作 O(1)。**因为 `rear` 始终指向队尾，无需遍历整个队列。
`pop(x, y)`	出队：移除 `front->next`（第一个数据节点），并返回其值。若删除后队列为空，需要让 `rear = front`。	**队头操作 O(1)。**使用头结点方便删除第一个数据节点，不需要移动 `front` 本身。保证空队列时 `front == rear`。
`destroy()`	释放队列中所有节点。	循环 `delete` 所有节点，防止**内存泄漏。**

2.4.4.2 BFS 函数

```
/**
 * @brief BFS 广度优先搜索
 * @param x, y 起点坐标
 * @return 无
 * 功能：从起点 (x, y) 开始，访问与其四邻域相连的所有 1，并标记为已访问。
 */
void BFS(int x, int y):
    Q.init()
        Q.push(x, y)
        Vis[x][y] = true
        当 Q 非空时:
            Q.pop(cx, cy)
            对 i 从 0 到 3:
                nx = cx + dx[i]
                ny = cy + dy[i]
                如果 nx, ny 在地图范围内 且 Map[nx][ny] == 1 且 Vis[nx][ny] == false:
                    Vis[nx][ny] = true
                    Q.push(nx, ny)
        Q.destroy()
```

2.4.4.3 主函数逻辑

```
int main():
    读入 n, m
    对 i 从 0 到 n-1:
        对 j 从 0 到 m-1:
            读入 Map[i][j]
    将 Vis 全部置为 0
    count = 0

    对 i 从 1 到 n-2:
        对 j 从 1 到 m-2:
            如果 Map[i][j] == 1 且 Vis[i][j] == false:
                count = count + 1
                BFS(i, j)

    输出 count
```

2.4.5 调试分析

1. 边界问题：

题目要求“仅在矩阵边缘联通的不算区域”，因此主循环应排除边界（i 从 1~（n-2）计数）。

2. pop 函数条件分析：

起初我在设计出队函数时没处理唯一节点情况，导致测试点出错，更正：若 pop 删除最后一个数据节点，则应使 front==rear。

3. BFS 核心思想：

本题的核心在于：从矩阵中每一个未被访问过的“1”出发，通过 BFS 遍历该点所在的整片连通区域，并在遍历过程中将所有访问到的点标记为已访问，从而确保同一片区域不会被重复统计。BFS 使用队列作为辅助数据结构，通过“先进先出”的方式实现从起点向四周逐层扩展的过程。具体步骤为：首先将起始坐标入队并标记访问；然后在循环中不断取出队头元素，依次检查其四个方向的邻接位置（上、下、左、右），若该邻点在矩阵范围内且值为 1 且尚未访问，则将其入队并标记访问；当队列为空时，说明该连通块已全部被搜索完毕，此时计数器加一。整个矩阵扫描结束后，计数器的值即为区域总数。

2.4.6 总结与体会

通过本题，初步接触了 BFS（广度优先搜索）的思想，了解到该方式使用队列进行实现，进一步加深对链队列的掌握。

2.5 队列中的最大值

2.5.1 题目描述

给定一个队列，有下列 3 个基本操作：

（1）Enqueue(v)：v 入队

（2）Dequeue()：使队首元素删除，并返回此元素

（3）GetMax()：返回队列中的最大元素

请设计一种数据结构和算法，让 GetMax 操作的时间复杂度尽可能地低。要求运行时间不超过一秒

2.5.2 基本要求

输入：

第 1 行 1 个正整数 n，表示队列的容量(队列中最多有 n 个元素)

接着读入多行，每一行执行一个动作。

若输入"dequeue"，表示出队，当队空时，输出一行“Queue is Empty”;否则，输出出队的元素；

若输入"enqueue m",表示将元素 m 入队,当队满时(入队前队列中元素已有 n 个),输出"Queue is Full", 否则, 不输出;

若输入"max",输出队列中最大元素, 若队空, 输出一行“Queue is Empty”。

若输入"quit",结束输入, 输出队列中的所有元素

对于 20%的数据, 有 0<=n<=100, |max(m) - min(m)|<=1e4;

对于 40%的数据, 有 0<=n<=6000, |max(m) - min(m)|<=1e6; (未优化 O(nm)程序运行时间略微超过一秒)

对于 100%的数据, 有 0<=n<=10000,|max(m) - min(m)|<=1e8;

对于每个测试点, 0x80000000 <= min(m) < max(m) <= 0x7fffffff

对于每个测试点, 操作的个数约为队列大小的 10 倍左右

输出:

多行, 分别是执行每次操作后的结果

2.5.3 数据结构设计

本实验采用三个结构体的设计, 实现单调队列。

```
// ValueQueue: 循环顺序队列（存数据）
struct ValueQueue {
    int *data;       // 动态分配数组：存储实际元素
    int front;       // 队头指针，指向队首元素的前一个位置
    int rear;        // 队尾指针，指向队尾元素位置（最后一个有效元素）
    int cap;         // 底层数组长度 = capacity + 1
};
```

```c
// MaxDeque: 单调双端队列（维护最大值候选）
/*
 * 这是一个双端队列，用于保证单调递减序列。
 * 队头元素 (MD_Front) 始终是当前队列的最大值。
 */
struct MaxDeque {
    int *data;
    int front;
    int rear;
    int cap;
};
```

```c
// MaxQueue: 组合上述两个结构体，最高层
struct MaxQueue {
    ValueQueue vq;      // 存放值的队列
    MaxDeque md;        // 存放最大值候选的单调队列
    int capacity;
};
```

2.5.4 功能实现

2.5.4.1 Enqueue 函数

```
/**
 * @brief Enqueue 操作（均摊 O(1)）
 * @param mq 队列引用
 * @param val 待插入值
 */
```cpp
MQ_Enqueue(mq, val):
    // Step1: 保证单调递减，移除所有尾部小于 val 的元素
    while (md 非空 且 MD_Back(md) < val):
        MD_PopBack(md)
    // Step2: 插入 val 到 md 尾部
    MD_PushBack(md, val)
    // Step3: 插入 val 到 vq 尾部
    VQ_Push(vq, val)
```

2.5.4.2 Dequeue 函数
```

```
/**
 * @brief Dequeue 操作（均摊 O(1)）
 * @param mq 队列引用
 * @param ok 输出参数，表示是否成功出队
 * @return int 返回被删除的元素
 */
```cpp
MQ_Dequeue(mq, ok):
    if VQ_IsEmpty(vq):
        ok ← false
        return INT_MIN

    frontVal ← VQ_Pop(vq)
    if MD_Front(md) == frontVal:
        MD_PopFront(md)
    ok ← true
    return frontVal
```

2.5.4.3 GetMax 函数
```

```
/**
 * @brief GetMax 操作（O(1)）
 * @param mq 队列引用
 * @param ok 输出参数，表示是否成功
 * @return int 当前最大值
 */
```cpp
MQ_GetMax(mq, ok):
    if VQ_IsEmpty(vq):
        ok ← false
        return INT_MIN
    ok ← true
    return MD_Front(md)
```

2.5.4.4 初始化最大值函数
```

```
 * @brief 初始化最大值队列 MaxQueue
 * @param mq 队列引用
 * @param n 容量
 */
```cpp
MQ_Init(mq, n):
    mq.capacity ← n
    VQ_Init(mq.vq, n)
    MD_Init(mq.md, n)
```

2.5.4.5 打印输出函数

```
 * @brief 获取当前队列中所有元素（用于 quit 输出）
 */
```cpp
MQ_GetAll(mq):
    return VQ_GetAll(vq)
```

2.5.4.6 ValueQueue 相关函数简略说明

1. `ValueQueue`（实际数据队列）

函数功能	函数原型 / 伪代码说明
初始化	`void VQ_Init(ValueQueue &q, int capacity)` 初始化队列，动态分配数组，设置 `front = rear = 0`。
判断空	`bool VQ_IsEmpty(const ValueQueue &q)` 返回 `q.front == q.rear`。
判断满	`bool VQ_IsFull(const ValueQueue &q)` 返回 `(q.rear + 1) % q.cap == q.front`。
入队	`void VQ_Push(ValueQueue &q, int val)` 队尾入队：`rear = (rear + 1) % cap; data[rear] = val`。
出队	`int VQ_Pop(ValueQueue &q)` 队头出队并返回被移除的元素值：`front = (front + 1) % cap`。
取队头	`int VQ_Front(const ValueQueue &q)` 返回队头元素：`data[(front + 1) % cap]`，不修改队列。
获取所有元素	`std::vector<int> VQ_GetAll(const ValueQueue &q)` 按 FIFO 顺序返回队列中所有元素的列表。

2.5.4.7 MaxDeque 相关函数简略说明

2. `MaxDeque`（单调双端队列）

函数功能	函数原型 / 伪代码说明
初始化	`void MD_Init(MaxDeque &d, int capacity)` 初始化双端队列，动态分配数组。
判断空	`bool MD_IsEmpty(const MaxDeque &d)` 返回 `d.front == d.rear`。
尾部入队	`void MD_PushBack(MaxDeque &d, int val)` 尾部插入元素 `val`。
尾部出队	`void MD_PopBack(MaxDeque &d)` 尾部删除元素（用于 `MQ_Enqueue` 淘汰小元素）。
头部出队	`void MD_PopFront(MaxDeque &d)` 头部删除元素（用于 `MQ_Dequeue` 同步最大值）。
取队头	`int MD_Front(const MaxDeque &d)` 返回队头元素值（即当前最大值）。
取队尾	`int MD_Back(const MaxDeque &d)` 返回队尾元素值。

2.5.5 调试分析

1. 双队列同步问题：

起初我在 MD_Dequeue 函数中没有保证 MaxDeque 和 ValueQueue 同步更新，导致错误，修正加上条件：只有当 ValueQueue 出队的元素等于 MaxDeque 的队头元素时，才执行 MD_PopFront。

2. 循环数组边界处理：

ValueQueue 和 MaxDeque 的双指针的移动必须使用模运算%cap，确保循环队列的正确性。且在判断队列的空与满时要注意正确处理双指针的相对位置。与链栈不同，循环指针中 front 指针本身不存储有效数据元素。

3. 如何实现时间复杂度的优化（本题难点）：

核心在于 Enqueue 中的均摊思想，while 循环在最差情况下执行 n 次，但是执行

后的元素不会再进入 while 循环，因此对于 m 个元素，操作均摊下来是 O（1）的时间复杂度。这种时间复杂度上的优化是通过开辟额外的 MaxDeque 队列牺牲一部分空间来实现的。

2.5.6 总结和体会

通过本题，我进一步加深了对循环队列的理解与操作。同时接触到“双端队列”的思想，进一步加深“空间换时间”算法的理解，了解了均摊时间复杂度这一概念。

3. 实验总结

通过本次实验，我进一步体会到选择合适的数据结构对程序设计效率和正确性的重要性。在解决实际问题时，若需要临时存储或回溯数据，栈（LIFO）结构是理想的选择；而当任务需要按顺序处理或逐层扩展时，队列（FIFO）结构则更为合适。例如在图论算法中，深度优先搜索（DFS）通常借助栈实现，而广度优先搜索（BFS）则使用队列结构来实现逐层访问。

在实验过程中，通过分别实现顺序栈、链栈、循环队列及链队列，加深了对其基本操作（入栈/出栈、入队/出队、判空、初始化等）的理解。特别是链式结构在动态存储管理中的优势显著，能够有效避免顺序存储结构中常见的“溢出”与“空间浪费”问题。